

Aula 06 — Gatilhos, cursores, estruturas de controle e tratamento de exceções

Disciplina: Banco de Dados II

Curso: Bacharelado em Sistemas de Informação

Semestre: 2026.2

Professor: Alysson M. Bruno

Data: 19/09/2026

Turma no Google Classroom: <https://classroom.google.com/c/ODY5MzE1NDkyODUx>

Turma no iximiuz Labs: <https://labs.iximiuz.com/trainings/aulas-bd2-2026-2-51d1c714>

Objetivos

Ao concluir este estudo, você deverá conseguir:

- relacionar o conteúdo das Aulas 01 a 05 e identificar o limite da integridade declarativa;
 - empregar IF, CASE, LOOP, WHILE e FOR em rotinas PL/pgSQL;
 - capturar erros com EXCEPTION e emitir mensagens próprias com RAISE;
 - interpretar os valores de SQLSTATE e SQLERRM;
 - percorrer o resultado de uma consulta com cursor implícito e com cursor explícito;
 - justificar quando o SQL de conjunto é preferível ao cursor;
 - escrever a função gatilho e o comando CREATE TRIGGER que a associa a uma tabela;
 - distinguir BEFORE de AFTER e FOR EACH ROW de FOR EACH STATEMENT;
 - inspecionar, desabilitar e remover gatilhos existentes;
 - avaliar criticamente os riscos de deslocar regras de negócio para gatilhos.
-

1. Retomada: o caminho das Aulas 01 a 05

Esta aula não começa do zero. Ela fecha um arco que vem sendo construído desde o primeiro encontro, e vale a pena percorrê-lo antes de seguir adiante.

1.1 Uma frase por aula

Aula	O que ficou estabelecido
01	O SGBD guarda o dado e responde por ele; no modelo relacional, tudo é tabela.
02	O banco recusa o dado errado por conta própria, por meio de restrições.
03	Sabemos formular perguntas complexas com junções, subconsultas e conjuntos.
04	O servidor é um programa independente, que executamos em contêiner.
05	A regra de negócio pode ser armazenada dentro do próprio banco.

A Aula 06 acrescenta a última peça: a regra que **o próprio servidor dispara**, sem que nenhuma aplicação precise se lembrar de chamá-la.

1.2 O que cada aula deixou pronto para hoje

Da Aula 01 vem o vocabulário. Gatilho é uma regra que age sobre **linhas** de **tabelas**; cursor percorre **linhas**; exceção interrompe a alteração de **linhas**. Sem o modelo relacional, nada disso tem sentido.

Da Aula 02 vem a integridade declarativa. Revise o quadro, porque a Seção 1.3 parte exatamente dele:

Restrição	O que garante	O que enxerga
PRIMARY KEY	identificação única da linha	a própria coluna
FOREIGN KEY	referência válida para outra tabela	a coluna e a tabela referenciada
NOT NULL	preenchimento obrigatório	a própria coluna
CHECK	condição sobre a linha	apenas a linha nova
UNIQUE	ausência de repetição	a coluna em toda a tabela

Declarativa significa que você **descreve** a regra e o servidor se encarrega de cumpri-la, sem que ninguém escreva o procedimento de verificação.

Da Aula 03 vem a agregação que reaparece aqui:

```
SELECT SUM(quantidade * preco_unitario)
  FROM item_pedido
 WHERE pedido_id = 1;
```

Na Aula 05 esse comando virou a função `fn_total_pedido`. Nesta aula, ele passará a ser executado sozinho, por um gatilho, sempre que um item de pedido mudar.

Da Aula 04 vem o ambiente:

```
podman run -d --name bd2-pg -e POSTGRES_PASSWORD=bd2 \
-e POSTGRES_DB=loja docker.io/library/postgres:17
```

Da Aula 05 vem a estrutura da rotina — cabeçalho, retorno, linguagem, corpo delimitado por cifrões, DECLARE, BEGIN e END — além das convenções de nome da disciplina: **fn_** para função, **sp_** para procedimento, **p_** para parâmetro e **v_** para variável. Hoje acrescentamos duas: **c_** para cursor e **fn_tg_** para função gatilho.

1.3 O limite da integridade declarativa

A loja decidiu que nenhum preço pode subir mais de 50% de uma só vez. É uma regra de integridade, e a primeira tentativa é natural:

```
ALTER TABLE produto
ADD CONSTRAINT preco_max CHECK (preco <= preco_antigo * 1.5);
```

```
ERROR: column "preco_antigo" does not exist
```

A CHECK não falhou por detalhe de sintaxe: ela falhou porque **enxerga apenas a linha nova**. O valor anterior não existe no universo dela.

Generalizando, há quatro necessidades frequentes que nenhuma restrição declarativa atende:

Necessidade	Por que a restrição não resolve
comparar o valor antigo com o novo	a restrição só recebe a linha nova
registrar quem alterou o quê e quando	restrição não escreve em outra tabela
manter um valor derivado atualizado	restrição não altera dados
corrigir o dado antes de gravar	restrição apenas aceita ou recusa

Ponto-chave: as quatro necessidades têm a mesma resposta — **gatilho**. Mas o gatilho é uma rotina, e uma rotina precisa saber decidir, repetir e tratar erro. Por isso as três primeiras seções desta aula são ferramentas, e a quarta é o objetivo.

1.4 O esquema desta aula

Continuamos na loja da Aula 05, com três acréscimos:

Novidade	Finalidade nesta aula
cliente.email, com UNIQUE	estudar a exceção unique_violation
pedido.total	manter um valor derivado por gatilho
tabela auditoria_produto	guardar a trilha de alterações

Um esquema crescer junto com as regras do negócio é o comportamento normal de um sistema em produção, não um defeito de projeto.

2. Estruturas de controle em PL/pgSQL

O SQL é uma linguagem **declarativa**: você descreve o resultado desejado e o servidor decide como obtê-lo. Isso é excelente para consultar e péssimo para expressar uma sequência de decisões. O PL/pgSQL acrescenta ao SQL exatamente o que falta: **decidir** e **repetir**.

2.1 A decisão com IF

```
IF condição1 THEN
    comandos;
ELSIF condição2 THEN
    comandos;
ELSE
    comandos;
END IF;
```

Três detalhes derrubam quem está começando:

1. a palavra é **ELSIF**, sem o “E” — nem ELSEIF, nem ELSE IF;
2. o bloco encerra com **END IF;**, com espaço e ponto e vírgula;
3. o primeiro THEN verdadeiro vence, e as condições seguintes nem são avaliadas.

O exemplo da aula calcula o frete por faixa de valor:

```
CREATE OR REPLACE FUNCTION fn_faixa_frete(p_total numeric)
RETURNS numeric
LANGUAGE plpgsql
IMMUTABLE
AS $$
DECLARE
    v_frete numeric;
BEGIN
    IF p_total >= 500 THEN
        v_frete := 0;
    ELSIF p_total >= 200 THEN
        v_frete := 15.00;
    ELSE
        v_frete := 30.00;
    END IF;

    RETURN v_frete;
END;
$;
```

```
SELECT fn_faixa_frete(600), fn_faixa_frete(300), fn_faixa_frete(50);
```

```
fn_faixa_frete | fn_faixa_frete | fn_faixa_frete
-----+-----+-----
              0 |          15.00 |          30.00
```

Pergunta de verificação: por que `fn_faixa_frete(500)` devolve 0, e não 15.00? Porque `500 >= 500` é verdadeiro e é testado primeiro. A **ordem dos degraus faz parte da regra** — inverter as duas primeiras condições mudaria o comportamento sem produzir erro algum.

2.2 A decisão com CASE

O CASE expressa a mesma escolha em formato de tabela e, diferentemente do IF, cabe dentro de um SELECT:

```
CREATE OR REPLACE FUNCTION fn_situacao_estoque(p_estoque integer)
RETURNS text
LANGUAGE plpgsql
IMMUTABLE
AS $$
BEGIN
    RETURN CASE
        WHEN p_estoque = 0 THEN 'esgotado'
        WHEN p_estoque <= 5 THEN 'crítico'
        WHEN p_estoque <= 20 THEN 'normal'
        ELSE 'folgado'
    END;
END;
$$;
```

Use CASE quando a decisão produz **um valor**; use IF quando ela dispara **vários comandos**.

Atenção: um CASE do PL/pgSQL sem ELSE provoca ERROR: case not found quando nenhuma condição é satisfeita. Adote como regra pessoal terminar todo CASE com ELSE, ainda que seja ELSE NULL.

2.3 Os laços

Forma	Quando usar
LOOP ... END LOOP	repete indefinidamente; alguém precisa mandar parar
WHILE cond LOOP	testa a condição antes de cada volta
FOR i IN 1..n LOOP	número de voltas conhecido
FOR r IN SELECT ... LOOP	uma volta por linha do resultado (Seção 4)

O exemplo com WHILE calcula em quantas parcelas um total cabe, respeitando um valor mínimo e o teto de doze vezes:

```
CREATE OR REPLACE FUNCTION fn_parcelas(  
    p_total      numeric,  
    p_valor_minimo numeric DEFAULT 100  
)  
RETURNS integer  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_n integer := 12;  
BEGIN  
    WHILE v_n > 1 AND p_total / v_n < p_valor_minimo LOOP  
        v_n := v_n - 1;  
    END LOOP;  
  
    RETURN v_n;  
END;  
$$;
```

```
SELECT fn_parcelas(1200), fn_parcelas(250), fn_parcelas(80);
```

fn_parcelas	fn_parcelas	fn_parcelas
12	2	1

Observe as duas saídas do laço: a condição $p_total / v_n < p_valor_minimo$, que é a regra do negócio, e a condição $v_n > 1$, que é a **garantia de término**. Um valor de R\$ 80,00 nunca alcança a parcela mínima de R\$ 100,00; sem a segunda condição, o laço subtrairia indefinidamente. **Todo laço precisa de uma saída garantida.**

No FOR com contador, a variável de controle **não é declarada**: ela nasce no próprio comando e só existe dentro dele.

```

CREATE OR REPLACE PROCEDURE sp_mostrar_parcelas(
    p_total numeric,
    p_vezes integer
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_valor numeric := round(p_total / p_vezes, 2);
BEGIN
    FOR v_i IN 1..p_vezes LOOP
        RAISE NOTICE 'Parcela %/% ..... R$ %', v_i, p_vezes, v_valor;
    END LOOP;
END;
$$;

```

```
CALL sp_mostrar_parcelas(900, 3);
```

```

NOTICE:  Parcela 1/3 ..... R$ 300.00
NOTICE:  Parcela 2/3 ..... R$ 300.00
NOTICE:  Parcela 3/3 ..... R$ 300.00
CALL

```

O RAISE NOTICE escreve na tela sem interromper a execução. É o instrumento de depuração mais simples e mais usado do PL/pgSQL: cada % da mensagem é substituído por um argumento, na ordem em que aparecem.

2.4 EXIT, CONTINUE e FOUND

Recurso	Efeito
EXIT	abandona o laço imediatamente
EXIT WHEN condição	abandona o laço quando a condição for verdadeira
CONTINUE	interrompe a volta atual e passa para a próxima
FOUND	variável booleana: o comando anterior encontrou alguma linha?

A variável FOUND é preenchida automaticamente pelo servidor após um SELECT ... INTO, um UPDATE, um DELETE ou um FETCH. É a forma idiomática de verificar se o registro procurado existe:

```

SELECT estoque INTO v_estoque FROM produto WHERE id = p_produto_id;

IF NOT FOUND THEN
    RAISE EXCEPTION 'Produto % não encontrado.', p_produto_id;
END IF;

```

FOUND reaparece na Seção 4, como condição de parada do cursor explícito.

3. Tratamento de exceções

3.1 O que acontece sem tratamento

Um erro dentro de uma rotina **aborta o comando inteiro**. O que já havia sido alterado por aquele comando é desfeito e a mensagem técnica do servidor chega a quem chamou. Em muitos casos é exatamente o comportamento desejado — um estoque negativo não deve ser gravado de jeito nenhum. Em outros, não: o programa que gera um relatório com mil divisões não deveria parar porque uma delas tem divisor zero.

3.2 O bloco EXCEPTION

```
BEGIN
    -- comandos que podem falhar
EXCEPTION
    WHEN condição THEN
        -- o que fazer diante desse erro específico
    WHEN OTHERS THEN
        -- qualquer outro erro
END;
```

Três consequências ao mesmo tempo:

1. o erro **não sobe** para quem chamou a rotina;
2. as alterações feitas **dentro do bloco** antes da falha são desfeitas;
3. a execução **continua** a partir do tratador escolhido.

```
CREATE OR REPLACE FUNCTION fn_dividir_seguro(
    p_dividendo numeric,
    p_divisor    numeric
)
RETURNS numeric
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN round(p_dividendo / p_divisor, 2);
EXCEPTION
    WHEN division_by_zero THEN
        RAISE NOTICE 'Divisor igual a zero; devolvendo NULL.';
        RETURN NULL;
END;
$$;
```

```
SELECT fn_dividir_seguro(10, 4);
SELECT fn_dividir_seguro(10, 0);
```

```
fn_dividir_seguro
-----
                2.50

fn_dividir_seguro
-----

NOTICE: Divisor igual a zero; devolvendo NULL.
```

A segunda chamada devolveu NULL — a coluna aparece vazia — e a consulta não foi interrompida.

3.3 As condições mais frequentes

Condição	Situação
division_by_zero	divisão por zero
unique_violation	valor repetido em coluna UNIQUE
foreign_key_violation	referência para linha inexistente
not_null_violation	NULL em coluna NOT NULL
check_violation	valor recusado por uma CHECK
no_data_found	SELECT ... INTO STRICT sem resultado
too_many_rows	SELECT ... INTO STRICT com mais de uma linha
OTHERS	qualquer erro não listado nos tratadores anteriores

São as mesmas violações estudadas na Aula 02, agora observadas de dentro da rotina. Vale o registro: a integridade continua sendo garantida pela restrição; o tratador apenas decide **o que fazer** quando ela é acionada.

3.4 SQLSTATE e SQLERRM

Dentro do tratador, duas variáveis ficam disponíveis:

Variável	Conteúdo
SQLSTATE	o código do erro, com cinco caracteres
SQLERRM	a mensagem de erro produzida pelo servidor

O código do SQLSTATE segue o padrão SQL e é portátil: 23505 identifica violação de unicidade em qualquer SGBD que siga a norma, e 22012, divisão por zero. Registrar o código em um log é mais útil do que registrar a mensagem, que varia com o idioma e com a versão.

```

CREATE OR REPLACE PROCEDURE sp_cadastrar_cliente(
    p_nome    text,
    p_cidade text,
    p_email   text
)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO cliente (nome, cidade, email)
    VALUES (p_nome, p_cidade, p_email);

    RAISE NOTICE 'Cliente % cadastrado.', p_nome;
EXCEPTION
    WHEN unique_violation THEN
        RAISE NOTICE 'O e-mail % já pertence a outro cliente.', p_email;
        RAISE NOTICE 'Código do erro: % - %', SQLSTATE, SQLERRM;
END;
$$;

```

```

CALL sp_cadastrar_cliente('Diego Alves', 'Gurupi', 'diego@exemplo.com');
CALL sp_cadastrar_cliente('Outro Diego', 'Palmas', 'diego@exemplo.com');

```

```

NOTICE: Cliente Diego Alves cadastrado.
CALL
NOTICE: O e-mail diego@exemplo.com já pertence a outro cliente.
NOTICE: Código do erro: 23505 – duplicate key value violates unique constraint
"cliente_email_key"
CALL

```

A primeira mensagem é escrita para o atendente; a segunda, para o suporte técnico. E note o CALL ao final da segunda chamada: para quem chamou, o procedimento terminou normalmente, embora nada tenha sido gravado. Essa é uma decisão de projeto que precisa ser consciente — em um cadastro real, talvez fosse melhor deixar o erro subir.

3.5 RAISE: falar e interromper

Nível	Efeito
RAISE NOTICE	escreve a mensagem e continua
RAISE WARNING	idem, com destaque maior
RAISE EXCEPTION	escreve a mensagem e interrompe a execução

A cláusula USING acrescenta informações ao erro. A mais útil no material didático é HINT, que diz ao usuário **o que fazer**:

```

RAISE EXCEPTION 'Estoque insuficiente: há % unidades e foram pedidas %.',
    v_estoque, p_quantidade
    USING HINT = 'Reponha o estoque ou reduza a quantidade.';

```

O procedimento completo da aula:

```
CREATE OR REPLACE PROCEDURE sp_baixar_estoque(  
    p_produto_id integer,  
    p_quantidade integer  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_estoque integer;  
BEGIN  
    SELECT estoque INTO v_estoque  
    FROM produto  
    WHERE id = p_produto_id;  
  
    IF NOT FOUND THEN  
        RAISE EXCEPTION 'Produto % não encontrado.', p_produto_id  
        USING HINT = 'Confira a lista com SELECT id, nome FROM produto;';  
    END IF;  
  
    IF v_estoque < p_quantidade THEN  
        RAISE EXCEPTION 'Estoque insuficiente: há % unidades e foram pedidas %.',  
            v_estoque, p_quantidade  
        USING HINT = 'Reponha o estoque ou reduza a quantidade.';  
    END IF;  
  
    UPDATE produto  
    SET estoque = estoque - p_quantidade  
    WHERE id = p_produto_id;  
  
    RAISE NOTICE 'Baixa de % unidade(s) registrada no produto %.',  
        p_quantidade, p_produto_id;  
END;  
$$;
```

```
CALL sp_baixar_estoque(3, 1);  
SELECT id, nome, estoque FROM produto WHERE id = 3;  
CALL sp_baixar_estoque(3, 500);
```

NOTICE: Baixa de 1 unidade(s) registrada no produto 3.

id	nome	estoque
3	Monitor	3

ERROR: Estoque insuficiente: há 3 unidades e foram pedidas 500.

HINT: Reponha o estoque ou reduza a quantidade.

CONTEXT: PL/pgSQL function sp_baixar_estoque(integer,integer) line 15 at RAISE

A rotina parou **antes** do UPDATE: o estoque permanece em três unidades. Repare também na linha CONTEXT, que indica a rotina e a linha do código onde o erro foi emitido — é por ela que se localiza o

problema.

3.6 Três cuidados

Primeiro: WHEN OTHERS sozinho esconde defeito. Capturar qualquer erro e seguir em frente converte um defeito de programação em resultado errado silencioso. Se for capturar OTHERS, ao menos registre o SQLSTATE e o SQLERRM em algum lugar.

Segundo: o bloco com EXCEPTION tem custo. O PostgreSQL estabelece um ponto de salvamento a cada entrada em um bloco que tenha tratador de exceção, para poder desfazer o que foi alterado. Dentro de um laço de mil voltas, são mil pontos de salvamento. Quando possível, prefira validar antes com IF.

Terceiro: exceção não é desvio de fluxo. Se a condição pode ser testada com um IF — e quase sempre pode —, teste com IF. Reserve a exceção para o que é efetivamente excepcional.

4. Cursores

4.1 Conjunto e linha a linha

O SQL foi projetado para operar sobre **conjuntos**. Um único UPDATE altera quatro ou quatro milhões de linhas com o mesmo comando, e o otimizador escolhe a melhor forma de fazê-lo. Um **cursor**, ao contrário, é um marcador que caminha pelo resultado de uma consulta **uma linha por vez**.

O cursor é a ferramenta certa quando cada linha exige um tratamento próprio: emitir uma mensagem diferente para cada produto, chamar outra rotina para cada pedido, montar um relatório em que cada linha depende da anterior. Ele é a ferramenta errada para aplicar a mesma alteração a todas as linhas — nesse caso, um UPDATE resolve com menos código e muito mais velocidade.

4.2 Cursor implícito

A forma mais usada no dia a dia sequer menciona a palavra cursor:

```
FOR v_linha IN
    SELECT id, nome, preco FROM produto ORDER BY id
LOOP
    -- v_linha.id, v_linha.nome, v_linha.preco
END LOOP;
```

O FOR ... IN SELECT abre, percorre e fecha o cursor automaticamente. A variável v_linha é do tipo record e recebe a linha inteira; os campos são acessados com ponto. Quando as linhas acabam, o laço termina — não é preciso EXIT.

```

CREATE OR REPLACE PROCEDURE sp_reajustar_precos(p_percentual numeric)
LANGUAGE plpgsql
AS $$
DECLARE
    v_linha record;
    v_novo numeric;
    v_conta integer := 0;
BEGIN
    FOR v_linha IN
        SELECT id, nome, preco FROM produto ORDER BY id
    LOOP
        v_novo := round(v_linha.preco * (1 + p_percentual / 100), 2);

        UPDATE produto SET preco = v_novo WHERE id = v_linha.id;

        RAISE NOTICE '% : % -> %', v_linha.nome, v_linha.preco, v_novo;
        v_conta := v_conta + 1;
    END LOOP;

    RAISE NOTICE '% produto(s) reajustado(s).', v_conta;
END;
$$;

```

```
CALL sp_reajustar_precos(10);
```

```

NOTICE: Teclado : 120.00 -> 132.00
NOTICE: Mouse : 80.00 -> 88.00
NOTICE: Monitor : 900.00 -> 990.00
NOTICE: Cabo HDMI : 35.00 -> 38.50
NOTICE: 4 produto(s) reajustado(s).

```

Guarde um detalhe para a Seção 5.5: esta rotina executa **quatro comandos UPDATE**, um por volta do laço, e não um único comando que altera quatro linhas. A diferença é invisível no resultado e decisiva para os gatilhos.

4.3 Cursor explícito

O cursor explícito torna visíveis as quatro etapas que o FOR escondia:

```

CREATE OR REPLACE PROCEDURE sp_listar_estoque_baixo(p_limite integer DEFAULT 5)
LANGUAGE plpgsql
AS $$
DECLARE
    c_produto CURSOR (p_lim integer) FOR
        SELECT id, nome, estoque
        FROM produto
        WHERE estoque <= p_lim
        ORDER BY estoque, id;

    v_linha record;
BEGIN
    OPEN c_produto(p_limite);

    LOOP
        FETCH c_produto INTO v_linha;
        EXIT WHEN NOT FOUND;

        RAISE NOTICE 'Produto % (%) com % unidade(s) – situação: %',
            v_linha.id, v_linha.nome, v_linha.estoque,
            fn_situacao_estoque(v_linha.estoque);
    END LOOP;

    CLOSE c_produto;
END;
$$;

```

Etapa	Comando	Papel
1	DECLARE ... CURSOR FOR	associa o nome à consulta
2	OPEN	executa a consulta e posiciona o marcador
3	FETCH ... INTO	traz a próxima linha e atualiza FOUND
4	CLOSE	libera o cursor

```
CALL sp_listar_estoque_baixo(5);
```

```
NOTICE: Produto 3 (Monitor) com 3 unidade(s) – situação: crítico
CALL
```

Dois pontos merecem destaque. O cursor pode receber **parâmetro**, declarado entre parênteses e informado no OPEN. E o EXIT WHEN NOT FOUND é **obrigatório**: sem ele, o FETCH deixa de trazer linhas, mas o LOOP continua girando indefinidamente.

Note ainda que o procedimento chamou fn_situacao_estoque, criada na Seção 2.2. Rotinas armazenadas combinam-se entre si como qualquer função de programa.

4.4 A alternativa sem cursor

O mesmo resultado, em SQL de conjunto:

```

CREATE OR REPLACE FUNCTION fn_estoque_baixo(p_limite integer DEFAULT 5)
RETURNS TABLE (
    produto_id integer,
    nome        text,
    estoque     integer,
    situacao    text
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
        SELECT pr.id, pr.nome, pr.estoque, fn_situacao_estoque(pr.estoque)
        FROM produto pr
        WHERE pr.estoque <= p_limite
        ORDER BY pr.estoque, pr.id;
END;
$$;

```

Menos código, e o resultado vem como tabela, pronto para ser usado em outra consulta. A qualificação `pr.` é a mesma precaução da Aula 05: os nomes declarados em `RETURNS TABLE` viram variáveis dentro da função e entram em conflito com os nomes das colunas.

Situação	Escolha
a mesma alteração em todas as linhas	UPDATE direto
devolver linhas a quem chamou	RETURNS TABLE com RETURN QUERY
tratamento diferente por linha	FOR ... IN SELECT
necessidade de abrir, pausar e fechar com controle	cursor explícito

Regra de bolso: comece pelo SQL de conjunto. Recorra ao cursor apenas quando a lógica de cada linha realmente não couber em um comando.

5. Gatilhos

5.1 O problema que o gatilho resolve

Na Aula 05, a regra passou a morar dentro do banco. Mas ela só é aplicada **se alguém a chamar**:

```
CALL sp_baixar_estoque(3, 1);
```

Se o programador esquecer a chamada, se um sistema novo for escrito por outra equipe ou se alguém alterar a tabela diretamente pelo DBeaver, a regra simplesmente não acontece. O banco volta a depender da disciplina de quem o usa.

O **gatilho** (em inglês, *trigger*) elimina essa dependência: ele é disparado pelo **próprio servidor**, sempre que a tabela sofre o evento combinado. Não há como esquecer de chamá-lo nem como contorná-lo.

5.2 As duas peças

Um gatilho no PostgreSQL é sempre formado por duas peças distintas.

Peça 1 — a função gatilho:

```
CREATE OR REPLACE FUNCTION fn_tg_exemplo()  
RETURNS trigger  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    ...  
    RETURN NEW;  
END;  
$$;
```

Ela difere das funções da Aula 05 em dois pontos: **não recebe parâmetros** entre os parênteses e **devolve o tipo trigger**, que existe apenas para esta finalidade. Tentar declarar um parâmetro produz ERROR: trigger functions cannot have declared arguments.

Peça 2 — o comando que associa a função à tabela:

```
CREATE TRIGGER tg_produto_auditar  
AFTER INSERT OR UPDATE OR DELETE  
ON produto  
FOR EACH ROW  
EXECUTE FUNCTION fn_tg_auditar_produto();
```

Lido como frase: *depois de inserir, alterar ou excluir na tabela produto, para cada linha, execute esta função.*

A separação é intencional: a mesma função gatilho pode ser associada a várias tabelas ou a vários eventos, e é por isso que ela precisa descobrir em tempo de execução em que contexto está sendo chamada.

5.3 As variáveis especiais

Variável	Conteúdo
NEW	a linha como ficará (INSERT e UPDATE)
OLD	a linha como estava (UPDATE e DELETE)
TG_OP	'INSERT', 'UPDATE', 'DELETE' ou 'TRUNCATE'
TG_NAME	o nome do gatilho
TG_TABLE_NAME	o nome da tabela que disparou
TG_WHEN	'BEFORE', 'AFTER' ou 'INSTEAD OF'
TG_LEVEL	'ROW' ou 'STATEMENT'

Nenhuma delas é declarada: o servidor as entrega prontas.

OLD e NEW não existem em todas as operações, e este é o erro mais comum da matéria:

Operação	OLD	NEW
INSERT	não existe	a linha que está entrando
UPDATE	a linha antes	a linha depois
DELETE	a linha que sai	não existe

Ler NEW durante um DELETE **não produz erro** no PostgreSQL 11 ou mais novo (a versão do laboratório é a 17): NEW vale NULL, e a função segue adiante com valores vazios. Um registro de log, por exemplo, sai gravado com as colunas nulas, sem nenhum aviso — o que torna o engano mais difícil de perceber. O mesmo vale para OLD em um INSERT.

Observação: até o PostgreSQL 10, essa leitura interrompia o comando com ERROR: record "new" is not assigned yet, mensagem que ainda aparece em muitos tutoriais. A mudança veio na versão 11 (POSTGRES GLOBAL DEVELOPMENT GROUP, 2018): desde então, NEW é nulo nas operações DELETE e OLD é nulo nas operações INSERT.

Por isso, toda função gatilho que atende a mais de um evento **começa testando TG_OP**.

5.4 BEFORE e AFTER

	BEFORE	AFTER
Momento	antes de gravar	depois de gravar
Pode alterar NEW	sim	não tem efeito
Pode cancelar a linha	sim , devolvendo NULL	não
Enxerga o dado já gravado	não	sim
Vocação	corrigir e validar	registrar e propagar

Regra prática: para mexer no dado, BEFORE; para reagir ao dado, AFTER.

5.5 FOR EACH ROW e FOR EACH STATEMENT

```
UPDATE produto SET preco = preco * 1.1;    -- 4 produtos na tabela
```

Tipo	Disparos
FOR EACH ROW	4 — um por linha afetada
FOR EACH STATEMENT	1 — o comando é um só

No gatilho de comando não existem OLD nem NEW, porque não há uma linha em questão. FOR EACH STATEMENT é o padrão quando a cláusula é omitida, e um comando que não altera nenhuma linha ainda assim o dispara.

Agora recupere o detalhe da Seção 4.2: o procedimento `sp_reajustar_precos` executa quatro comandos UPDATE. Um gatilho FOR EACH STATEMENT associado a produto dispara, portanto, **quatro vezes** durante essa chamada — uma por comando, não uma por rotina.

5.6 A regra do valor de retorno

Tipo de gatilho	O que devolver
BEFORE ... FOR EACH ROW	NEW para prosseguir; NULL para cancelar a linha
BEFORE DELETE ... FOR EACH ROW	OLD para prosseguir; NULL para cancelar
AFTER ... FOR EACH ROW	o retorno é ignorado; use RETURN NULL
qualquer FOR EACH STATEMENT	o retorno é ignorado; use RETURN NULL

O cancelamento em um gatilho BEFORE ROW é **silencioso**: o comando responde INSERT 0 0 e nenhum erro é emitido. É um recurso legítimo — usado, por exemplo, para descartar registros duplicados em uma carga —, mas precisa ser documentado, sob pena de alguém passar horas procurando por que a linha não foi gravada.

5.7 Exemplo 1 — corrigir o dado antes de gravar

```
CREATE OR REPLACE FUNCTION fn_tg_normalizar_cliente()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.nome := btrim(NEW.nome);
    NEW.cidade := btrim(NEW.cidade);
    NEW.email := lower(btrim(NEW.email));

    IF NEW.nome = '' THEN
        RAISE EXCEPTION 'O nome do cliente não pode ser vazio.'
        USING HINT = 'Informe o nome completo do cliente.';
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER tg_cliente_normalizar
BEFORE INSERT OR UPDATE ON cliente
FOR EACH ROW
EXECUTE FUNCTION fn_tg_normalizar_cliente();
```

```
INSERT INTO cliente (nome, cidade, email)
VALUES (' Elena Rocha ', 'Palmas', ' ELENA@Exemplo.COM ');

SELECT id, nome, email FROM cliente ORDER BY id DESC LIMIT 1;
```

id	nome	email
6	Elena Rocha	elena@exemplo.com

Ninguém pediu para limpar o dado: o gatilho o fez para **qualquer programa** que grave nessa tabela. E a validação vale igualmente para todos:

```
INSERT INTO cliente (nome, cidade, email)
VALUES (' ', 'Palmas', 'vazio@exemplo.com');
```

```
ERROR:  O nome do cliente não pode ser vazio.
HINT:  Informe o nome completo do cliente.
CONTEXT:  PL/pgSQL function fn_tg_normalizar_cliente() line 8 at RAISE
```

A linha CONTEXT identifica o gatilho responsável. É o caminho para descobrir uma regra que ninguém documentou.

5.8 Exemplo 2 — a regra que a CHECK não alcança

Voltamos à pergunta da Seção 1.3:

```
CREATE OR REPLACE FUNCTION fn_tg_limitar_reajuste()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_variacao numeric;
BEGIN
    v_variacao := round((NEW.preco - OLD.preco) / OLD.preco * 100, 2);

    IF v_variacao > 50 THEN
        RAISE EXCEPTION
            'Reajuste de % por cento no produto % excede o limite de 50 por
cento.',
            v_variacao, OLD.nome
            USING HINT = 'Reajustes maiores exigem autorização da diretoria.';
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER tg_produto_limitar_reajuste
BEFORE UPDATE ON produto
FOR EACH ROW
WHEN (NEW.preco IS DISTINCT FROM OLD.preco)
EXECUTE FUNCTION fn_tg_limitar_reajuste();
```

Só o gatilho recebe OLD e NEW ao mesmo tempo — é essa a diferença em relação à CHECK.

A cláusula **WHEN** filtra o disparo: a função só é chamada se o preço realmente mudou. Um UPDATE que mexa apenas no estoque não a incomoda. Em uma tabela grande, é a diferença entre chamar a função um milhão de vezes e não chamá-la nenhuma. O operador `IS DISTINCT FROM` é preferível a `<>` porque compara corretamente mesmo quando um dos lados é NULL.

```
CALL sp_reajustar_precos(10);
UPDATE produto SET preco = preco * 2 WHERE id = 1;
```

```
NOTICE: Teclado : 120.00 -> 132.00
...
ERROR: Reajuste de 100.00 por cento no produto Teclado excede o limite de 50 por
cento.
HINT: Reajustes maiores exigem autorização da diretoria.
```

O procedimento da Seção 4.2 e o gatilho convivem sem que um saiba do outro: cada UPDATE do laço passa pela conferência.

5.9 Exemplo 3 — auditoria com TG_OP

```
CREATE OR REPLACE FUNCTION fn_tg_auditar_produto()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO auditoria_produto
            (produto_id, operacao, campo, valor_antigo, valor_novo)
        VALUES (NEW.id, TG_OP, 'registro', NULL,
            format('%s | R$ %s | %s un.', NEW.nome, NEW.preco, NEW.estoque));

    ELSIF TG_OP = 'UPDATE' THEN
        IF NEW.preco IS DISTINCT FROM OLD.preco THEN
            INSERT INTO auditoria_produto
                (produto_id, operacao, campo, valor_antigo, valor_novo)
            VALUES (NEW.id, TG_OP, 'preco', OLD.preco::text, NEW.preco::text);
        END IF;

        IF NEW.estoque IS DISTINCT FROM OLD.estoque THEN
            INSERT INTO auditoria_produto
                (produto_id, operacao, campo, valor_antigo, valor_novo)
            VALUES (NEW.id, TG_OP, 'estoque', OLD.estoque::text,
NEW.estoque::text);
        END IF;

    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO auditoria_produto
            (produto_id, operacao, campo, valor_antigo, valor_novo)
        VALUES (OLD.id, TG_OP, 'registro',
            format('%s | R$ %s | %s un.', OLD.nome, OLD.preco, OLD.estoque),
            NULL);
    END IF;

    RETURN NULL;
END;
$$;

CREATE TRIGGER tg_produto_auditar
AFTER INSERT OR UPDATE OR DELETE ON produto
FOR EACH ROW
EXECUTE FUNCTION fn_tg_auditar_produto();
```

Depois de executar a baixa de estoque da Seção 3.5 e o reajuste de 10% da Seção 4.2:

```
SELECT id, produto_id, operacao, campo, valor_antigo, valor_novo
FROM auditoria_produto ORDER BY id;
```

id	produto_id	operacao	campo	valor_antigo	valor_novo
1	3	UPDATE	estoque	4	3
2	1	UPDATE	preco	120.00	132.00
3	2	UPDATE	preco	80.00	88.00
4	3	UPDATE	preco	900.00	990.00
5	4	UPDATE	preco	35.00	38.50

Nenhuma aplicação escreveu nessa tabela: **o gatilho escreveu**. É esse o argumento central da auditoria por gatilho — ela não depende da boa vontade nem da memória de quem programa o sistema.

5.10 Exemplo 4 — gatilho de comando

```
CREATE OR REPLACE FUNCTION fn_tg_resumo_produto()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_quantos integer;
BEGIN
    SELECT count(*) INTO v_quantos FROM produto;

    RAISE NOTICE 'Comando % concluído em %; a tabela tem % produto(s).',
        TG_OP, TG_TABLE_NAME, v_quantos;

    RETURN NULL;
END;
$$;

CREATE TRIGGER tg_produto_resumo
AFTER UPDATE ON produto
FOR EACH STATEMENT
EXECUTE FUNCTION fn_tg_resumo_produto();
```

Executando o reajuste, o resultado confirma o que a Seção 5.5 antecipou:

```
NOTICE: Comando UPDATE concluído em produto; a tabela tem 4 produto(s).
NOTICE: Teclado : 120.00 -> 132.00
NOTICE: Comando UPDATE concluído em produto; a tabela tem 4 produto(s).
NOTICE: Mouse : 80.00 -> 88.00
...
```

Quatro disparos, um por comando. E observe a ordem: o gatilho AFTER do UPDATE é executado **antes** do RAISE NOTICE que vem depois dele no corpo do laço — porque o gatilho faz parte do comando UPDATE, e o comando termina primeiro.

5.11 Exemplo 5 — manter um valor derivado

```
CREATE OR REPLACE FUNCTION fn_tg_total_pedido()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_pedido integer;
BEGIN
    IF TG_OP = 'DELETE' THEN
        v_pedido := OLD.pedido_id;
    ELSE
        v_pedido := NEW.pedido_id;
    END IF;

    UPDATE pedido
        SET total = fn_total_pedido(v_pedido)
        WHERE id = v_pedido;

    IF TG_OP = 'UPDATE' AND OLD.pedido_id IS DISTINCT FROM NEW.pedido_id THEN
        UPDATE pedido
            SET total = fn_total_pedido(OLD.pedido_id)
            WHERE id = OLD.pedido_id;
    END IF;

    RETURN NULL;
END;
$$;

CREATE TRIGGER tg_item_total
AFTER INSERT OR UPDATE OR DELETE ON item_pedido
FOR EACH ROW
EXECUTE FUNCTION fn_tg_total_pedido();
```

A função `fn_total_pedido`, escrita na Aula 05, é reaproveitada sem alteração.

```
SELECT id, total FROM pedido WHERE id = 1;           -- 275.00
INSERT INTO item_pedido VALUES (1, 2, 1, 80.00);
SELECT id, total FROM pedido WHERE id = 1;           -- 355.00
DELETE FROM item_pedido WHERE pedido_id = 1 AND produto_id = 2;
SELECT id, total FROM pedido WHERE id = 1;           -- 275.00
```

Inseriu, subiu; apagou, voltou — sem nenhuma linha de programa fazendo a conta.

Dois pontos de projeto merecem atenção. O primeiro é o tratamento do `DELETE`, em que apenas `OLD` existe. O segundo é o caso raro do `UPDATE` que transfere o item para outro pedido: aí **dois** pedidos precisam ser recalculados, e esquecer disso deixaria o total do pedido de origem errado para sempre.

5.12 O gatilho vale do presente para a frente

Quando o gatilho `tg_item_total` foi criado, os pedidos já gravados continuavam com `total = 0`: o gatilho não existia quando eles foram inseridos.

```
UPDATE pedido p SET total = fn_total_pedido(p.id);
```

Este comando roda **uma única vez**, para acertar o passado.

Ponto-chave: criar o gatilho não corrige o histórico. Todo projeto que passa a manter um valor derivado precisa desse acerto inicial — e de conferir, depois, se ele foi mesmo aplicado.

5.13 Ordem de disparo

Quando dois ou mais gatilhos atendem ao mesmo evento na mesma tabela, o PostgreSQL os dispara em **ordem alfabética de nome** — não na ordem de criação.

```
CREATE TRIGGER tg_zz_segundo BEFORE UPDATE ON produto ... ;
CREATE TRIGGER tg_aa_primeiro BEFORE UPDATE ON produto ... ;
```

```
NOTICE: sou o gatilho A
NOTICE: sou o gatilho B
```

Se a ordem importar para a correção da regra, ela precisa estar codificada no nome — e isso costuma ser sinal de que os dois deveriam ser um só.

5.14 Inspecionar e manter

```
\d produto
```

```
Triggers:
  tg_produto_auditar AFTER INSERT OR DELETE OR UPDATE ON produto
    FOR EACH ROW EXECUTE FUNCTION fn_tg_auditar_produto()
  tg_produto_limitar_reajuste BEFORE UPDATE ON produto FOR EACH ROW
    WHEN (new.preco IS DISTINCT FROM old.preco)
    EXECUTE FUNCTION fn_tg_limitar_reajuste()
  tg_produto_resumo AFTER UPDATE ON produto FOR EACH STATEMENT
    EXECUTE FUNCTION fn_tg_resumo_produto()
```

Fora do `psql`, a mesma informação vem do catálogo padrão:

```
SELECT trigger_name, event_manipulation, action_timing, action_orientation
FROM information_schema.triggers
WHERE event_object_table = 'produto'
ORDER BY trigger_name;
```

Um gatilho associado a três eventos aparece em **três linhas** nessa visão, uma por evento.

Para desligar temporariamente — em uma carga de dados, por exemplo — e religar:

```
ALTER TABLE produto DISABLE TRIGGER tg_produto_limitar_reajuste;
-- carga, correção em massa, importação...
ALTER TABLE produto ENABLE TRIGGER tg_produto_limitar_reajuste;
```

E para remover:

```
DROP TRIGGER tg_produto_resumo ON produto;
```

Três observações: o DROP TRIGGER exige o nome da **tabela**; apagar a tabela apaga seus gatilhos, mas **não** apaga as funções gatilho; e desligar um gatilho é decisão de manutenção que precisa ser registrada, sob risco de nunca mais ser religado.

5.15 Onde o gatilho ajuda e onde atrapalha

Ajuda quando a regra não pode depender de quem está chamando:

- auditoria de alterações;
- normalização do dado na entrada;
- regras que comparam o valor antigo com o novo;
- manutenção de valores derivados (total, saldo, contador);
- integridade entre tabelas que a FOREIGN KEY não consegue expressar.

Atrapalha porque é invisível:

- **lógica escondida** — o UPDATE faz coisas que o código da aplicação não mostra;
- **custo por linha** — um gatilho de linha em um UPDATE de milhões de registros multiplica o trabalho por milhões;
- **cascata** — um gatilho que altera uma tabela com gatilho que altera outra pode produzir efeitos difíceis de prever, inclusive recursão;
- **depuração difícil** — o erro aparece longe de onde foi causado;
- **dependência de ordem** — a ordem alfabética é uma amarração frágil.

Precisa de...	Use
valor obrigatório, único ou dentro de uma faixa	NOT NULL, UNIQUE, CHECK
referência entre tabelas	FOREIGN KEY
comparar antigo com novo, auditar, derivar	gatilho
cálculo chamado explicitamente	função ou procedimento
regra de tela, fluxo ou mensagem ao usuário	a aplicação

Suba um degrau por vez. Só recorra ao gatilho quando a restrição declarativa não alcançar: ela é mais simples, mais rápida e, sobretudo, mais visível. E documente todo gatilho que criar — quem for manter o sistema daqui a dois anos não vai adivinhar que ele existe.

5.16 Uma nota sobre INSTEAD OF

Além de BEFORE e AFTER, existe um terceiro momento:

```
CREATE TRIGGER tg_vw_cliente
  INSTEAD OF INSERT ON vw_cliente
  FOR EACH ROW
  EXECUTE FUNCTION fn_tg_gravar_cliente();
```

Ele só funciona sobre **visões**, nunca sobre tabelas, e serve para tornar gravável uma visão que reúne dados de várias tabelas. É sempre FOR EACH ROW. Guarde o nome: visões são o assunto da Aula 11.

6. Hands-on: o laboratório em casa

O laboratório completo, com todos os comandos e resultados esperados, está no arquivo `laboratorio-bd2-aula06.md`. O resumo do que você precisa saber antes de começar está aqui.

6.1 O ambiente e a cota de tempo

O ambiente é o playground **BD2-AULA06** do iximiuz Labs:

<https://labs.iximiuz.com/playgrounds/BD2-AULA06-c705669c>

O PostgreSQL 17 já vem baixado e os três scripts já estão em `~/lab06`. Nada é instalado no seu computador.

A conta gratuita da plataforma oferece cerca de uma hora de ambiente por dia, e o ambiente é destruído ao fim da sessão. Nada do que você digitar fica guardado para o dia seguinte.

O laboratório foi desenhado para conviver com isso. Ele está dividido em dois blocos de aproximadamente trinta minutos, e **cada bloco começa do zero**: os três scripts voltam para ~/lab06 toda vez que a máquina sobe, e o Bloco B reconstrói o banco inteiro em três comandos, em menos de um minuto. Faça os dois blocos no mesmo dia ou um em cada dia, como preferir.

Há aqui uma lição que vale além da aula: **recomeçar do zero é barato quando o ambiente está em código**. É a mesma razão pela qual, em produção, o esquema do banco fica versionado junto com o sistema.

Três hábitos fazem diferença: leia o roteiro **antes** de iniciar a máquina, tenha os comandos à mão para digitar sem consultar, e **anote** cada resultado. O ambiente você perde; as anotações, não.

6.2 Bloco A — controle, exceções e cursores

```
podman run -d --name bd2-pg -e POSTGRES_PASSWORD=bd2 \
-e POSTGRES_DB=loja docker.io/library/postgres:17
podman exec -i bd2-pg psql -U postgres -d loja < ~/lab06/01_esquema.sql
podman exec -it bd2-pg psql -U postgres -d loja
```

Escreva, na ordem: fn_faixa_frete, fn_situacao_estoque, fn_parcelas, fn_dividir_seguro, sp_baixar_estoque, sp_reajustar_precos e sp_listar_estoque_baixo. Provoque erros de propósito: divida por zero e peça 500 unidades de um produto que tem três.

O arquivo ~/lab06/02_rotinas.sql contém todas essas rotinas prontas. Use-o para **conferir** cada uma **depois** de escrevê-la; executá-lo no início cria tudo de uma vez e elimina justamente a prática que a aula treina.

6.3 Bloco B — gatilhos

O bloco começa recarregando os três scripts, mesmo que você tenha acabado o Bloco A: o 01_esquema.sql apaga as tabelas antes de recriá-las, e é isso que devolve a loja ao estado original e faz os resultados baterem com o roteiro.

```
podman exec -i bd2-pg psql -U postgres -d loja < ~/lab06/01_esquema.sql
podman exec -i bd2-pg psql -U postgres -d loja < ~/lab06/02_rotinas.sql
podman exec -i bd2-pg psql -U postgres -d loja < ~/lab06/03_gatilhos.sql
podman exec -it bd2-pg psql -U postgres -d loja
```

Depois, investigue: rode \d produto e leia a seção Triggers; insira um cliente com espaços e maiúsculas e confira a normalização; tente dobrar um preço e leia a mensagem de erro **inteira**, inclusive o CONTEXT; conte os avisos do gatilho de comando durante o reajuste; leia a tabela auditoria_produto; acompanhe pedido.total ao inserir e apagar um item; por fim, desabilite, habilite e remova um gatilho.

Os gatilhos usam funções definidas no 02_rotinas.sql — entre elas fn_total_pedido —, e é por isso que os três scripts são carregados na ordem.

7. Erros comuns e como interpretá-los

Mensagem	Causa provável
case not found	CASE sem cláusula ELSE
record "new" is not assigned yet	leitura de NEW em um gatilho de DELETE — só até o PostgreSQL 10; da versão 11 em diante NEW vale NULL e não há erro
record "old" is not assigned yet	leitura de OLD em um gatilho de INSERT — só até o PostgreSQL 10; da versão 11 em diante OLD vale NULL e não há erro
function ... must return type trigger	faltou RETURNS trigger na função gatilho
trigger functions cannot have declared arguments	parâmetro declarado na função gatilho
syntax error at or near "EXECUTE"	escreveu FOR EACH sem ROW ou STATEMENT
division by zero	falta o tratador WHEN division_by_zero

Observação: omitir a cláusula inteira **não dá erro**. Sem FOR EACH ROW, o PostgreSQL cria o gatilho como FOR EACH STATEMENT, que é o padrão: ele dispara uma vez por comando, NEW e OLD valem NULL e um teste como IF NEW.valor > 10 nunca é verdadeiro. A regra simplesmente deixa de valer, sem aviso. Conferido no PostgreSQL 16: um INSERT acima do limite foi gravado normalmente.

Outros enganos frequentes, que não produzem mensagem clara:

- escrever ELSEIF ou ELSE IF em vez de ELSIF;
- esquecer END IF;;, END LOOP; ou END;;
- montar um cursor explícito sem EXIT WHEN NOT FOUND, gerando laço infinito;
- executar DROP TRIGGER sem informar ON tabela;
- esperar que a criação do gatilho corrija os dados antigos da tabela;
- criar um gatilho AFTER e tentar alterar NEW dentro dele.

Leia a mensagem de erro inteira. As linhas HINT e CONTEXT costumam conter mais informação útil do que a linha ERROR.

8. Estudo orientado pelo celular

Se você não dispõe de computador, é possível estudar assim:

1. leia os slides na ordem, do início ao fim — eles contêm o laboratório completo;
2. copie à mão, em um caderno, o esqueleto de uma função gatilho e do CREATE TRIGGER, escrevendo ao lado o nome de cada parte;

3. para cada rotina apresentada, escreva com suas palavras: o que ela recebe, o que ela faz e o que ela devolve;
4. preencha de memória a tabela de disponibilidade de OLD e NEW nas três operações e confira na Seção 5.3;
5. explique por que a regra dos 50% não pode ser escrita como CHECK;
6. antes de olhar cada slide “Testando...”, escreva qual resultado você espera;
7. responda ao questionário no Google Classroom.

Copiar código à mão ensina: obriga a ler cada caractere, que é exatamente o que o compilador faz.

9. Atividade de consolidação

Parte A — leitura de código

Explique, em uma frase cada, o papel de: ELSIF, EXIT WHEN NOT FOUND, SQLSTATE, TG_OP e RETURN NEW.

Parte B — escrita

Escreva um gatilho que **impeça a inclusão de item em pedido cuja situação seja 'fechado'**. Decida e justifique: BEFORE ou AFTER? ROW ou STATEMENT? Em qual tabela ele deve ser criado?

Parte C — análise

Um colega criou o gatilho tg_item_total e reclamou de que os totais dos pedidos antigos continuavam zerados. Explique o que aconteceu e qual comando resolve.

Parte D — análise crítica

A equipe de desenvolvimento propôs transferir **todas** as regras de negócio do sistema para gatilhos, alegando que assim nenhuma seria esquecida. Aponte dois argumentos a favor e dois argumentos contra essa decisão.

Parte E — questões objetivas

Responda às dez questões do arquivo `exercícios-bd2-aula06.md` no Google Classroom.

10. Síntese

- O PL/pgSQL acrescenta ao SQL a capacidade de **decidir** (IF, CASE) e **repetir** (LOOP, WHILE, FOR).
- Um bloco BEGIN ... EXCEPTION ... END captura o erro, desfaz o que ele alterou e permite continuar; SQLSTATE e SQLERRM descrevem o que houve.
- RAISE NOTICE informa; RAISE EXCEPTION interrompe; USING HINT orienta quem lê o erro.

- O **cursor** percorre o resultado linha a linha, mas o SQL de conjunto vem primeiro sempre que resolver.
 - Um **gatilho** é formado por uma função RETURNS trigger e pelo comando CREATE TRIGGER que a associa a uma tabela, a um evento e a um momento.
 - BEFORE corrige e pode cancelar; AFTER registra e propaga. FOR EACH ROW dispara por linha; FOR EACH STATEMENT, por comando.
 - OLD não existe em INSERT; NEW não existe em DELETE.
 - O gatilho vale **do momento em que é criado para a frente**, e é invisível para quem executa o comando — por isso, documente.
 - Na próxima aula, veremos o que acontece quando várias dessas alterações precisam valer **todas ou nenhuma**: as **transações** e as propriedades ACID.
-

Referências

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2019.

IXIMIUZ LABS. **Playgrounds**. 2026. Disponível em: <https://labs.iximiuz.com/playgrounds>. Acesso em: 16 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 11 release notes**. 2018. Disponível em: <https://www.postgresql.org/docs/release/11.0/>. Acesso em: 27 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 17 documentation — Chapter 39: Triggers**. 2026. Disponível em: <https://www.postgresql.org/docs/17/triggers.html>. Acesso em: 16 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 17 documentation — Chapter 43: PL/pgSQL — SQL Procedural Language**. 2026. Disponível em: <https://www.postgresql.org/docs/17/plpgsql.html>. Acesso em: 16 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 17 documentation — Appendix A: PostgreSQL Error Codes**. 2026. Disponível em: <https://www.postgresql.org/docs/17/errcodes-appendix.html>. Acesso em: 16 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 17 documentation — CREATE TRIGGER**. 2026. Disponível em: <https://www.postgresql.org/docs/17/sql-createtrigger.html>. Acesso em: 16 set. 2026.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. **Sistema de banco de dados**. 7. ed. Rio de Janeiro: LTC, 2020. E-book. Disponível em: Minha Biblioteca.